Volterra on FPGA

Dot Product/Product Function

By: Helly Bera

Weekly Work Summary - Week 1, 2

- Focused on understanding the matlab code and running it on my PC.
- Focused on understanding dot product and product functions.

- Obtained the values from the MATLAB code.
- Wrote the dot product function and tested it.

Weekly Work Summary - Week 3, 4

Product Function:

- Used various parameters and flattened array inputs and outputs.
- Initially used bitwidth of individual values as 16, but this resulted in overflow when testing using experimental values, so changed it to 32.
- Used two different inputs for the d2 matrix(120x2 size).

Dot Product Function

```
% Dot product with 2nd order kernel
z2_val = (prod2_terms)' * h2;
```

- Used various parameters and flattened array inputs and outputs.
- Unpack the input and output arrays in variables declared as wire type (since they can be multidimensional matrices).
- Initially used bitwidth of individual values as 16, but this resulted in overflow error when testing using experimental values, so changed it to 32.

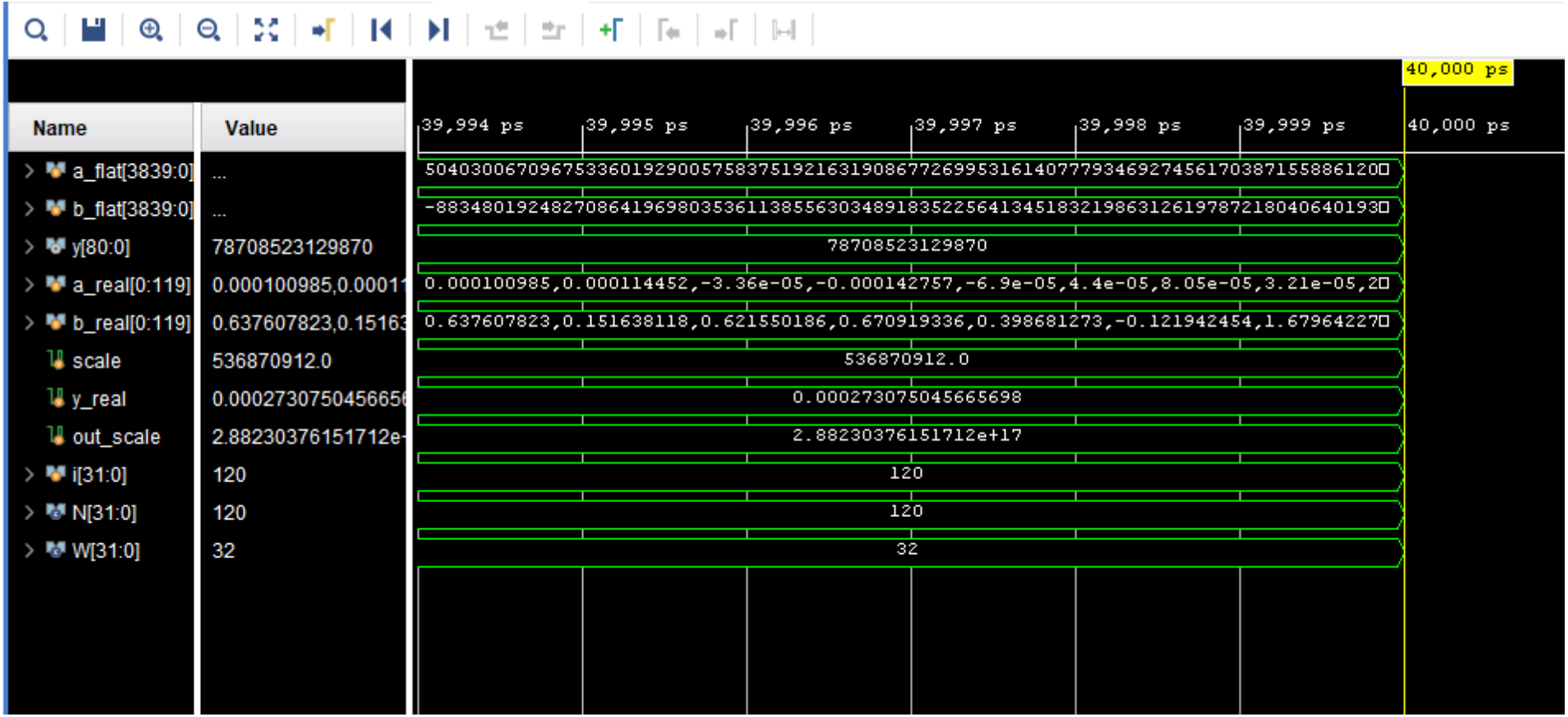
```
Starting Dot Product Testbench...
--------------------------------------------------
Dot product = 0.000273075046
Actual value from MATLAB = 2.730637760187415e-04
--------------------------------------------------
$finish called at time : 40 ns : File "C:/Users/Student/dot_product/dot_product.srcs/sim_1/new/dot_product_tb.v" Line 306
INFO: [USF-XSim-96] XSim completed. Design snapshot 'dot_product_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:03 ; elapsed = 00:00:06 . Memory (MB): peak = 1291.285 ; gain = 0.000
```

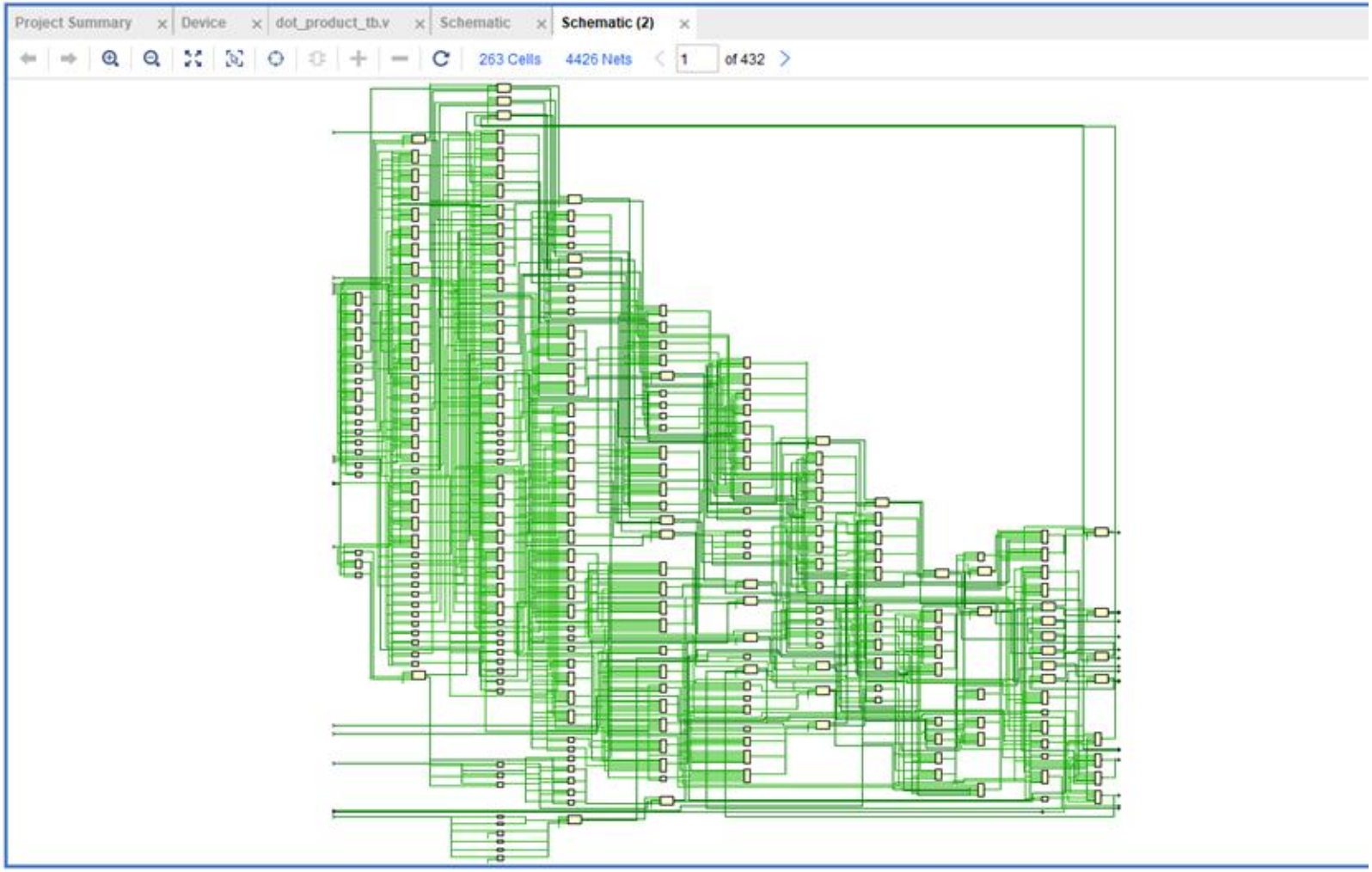
Dot product = 0.000273075046

Actual value from MATLAB = 2.730637760187415e-04

Absolute Error = $1.12699812585 * 10^{-8}$

Relative Error = $4.13 * 10^{-5}$


40,000 ps
Name
Value
39,994 ps
39,995 ps
39,996 ps
39,997 ps
39,998 ps
39,999 ps
40,000 ps
a_flat[3839:0]
...
b_flat[3839:0]
...
y[80:0]
78708523129870
a_real[0:119]
0.000100985,0.00011
b_real[0:119]
0.637607823,0.15163
scale
536870912.0
y_real
0.0002730750456656
out_scale
2.88230376151712e-
i[31:0]
120
N[31:0]
120
W[31:0]
32
5040300670967533601929005758375192163190867726995316140777934692745617038715588612 0
-8834801924827086419698035361138556303489183522564134518321986312619787218040640193 0
78708523129870
0.000100985,0.000114452,-3.36e-05,-0.000142757,-6.9e-05,4.4e-05,8.05e-05,3.21e-05,2 0
0.637607823,0.151638118,0.621550186,0.670919336,0.398681273,-0.121942454,1.67964227 0
536870912.0
0.000273075045665698
2.88230376151712e+17
120
120
32


Project Summary
Device
dot_product_tb.v
Schematic
Schematic (2)
263 Cells
4426 Nets
1
of 432

Tcl Console | Messages | Log | Reports | Design Runs | **Utilization**

- Hierarchy
- Summary
- Slice Logic
  - Slice LUTs (>100%)
    - LUT as Logic (>100%)
- Memory
- DSP
  - DSPs (100%)
    - DSP48E1 only
- IO and GT Specific
  - Bonded IOB (>100%)
- Clocking
- Specific Feature
- Primitives

Hierarchy

Name	Slice LUTs (63400)	DSPs (240)	Bonded IOB (210)
N dot_product	83898	240	7761

utilization_1

Product Function:

- Tested it using integer values.
- Then, tested it on experimental values of inputs : read them using a python script, stored them in the required format in a text file, and pasted in the testbench.

- Handled floating-point values by converting them into scaled integers (fixed-point), performing integer arithmetic in hardware, and scaling the result back for comparison.

Product Function

```
% Calculate 2nd order terms
prod2_terms = prod(yy(d{2}),2);
```

- yy → 15x1 double (column vector)
- d → 1x3 cell array
- d{2} is a 120x2 matrix containing index values from 1 to 15.
- yy is 15x1, so indexing it gives a row vector if multiple indices are used.

Product Function

```
% Calculate 2nd order terms
prod2_terms = prod(yy(d{2}),2);
```

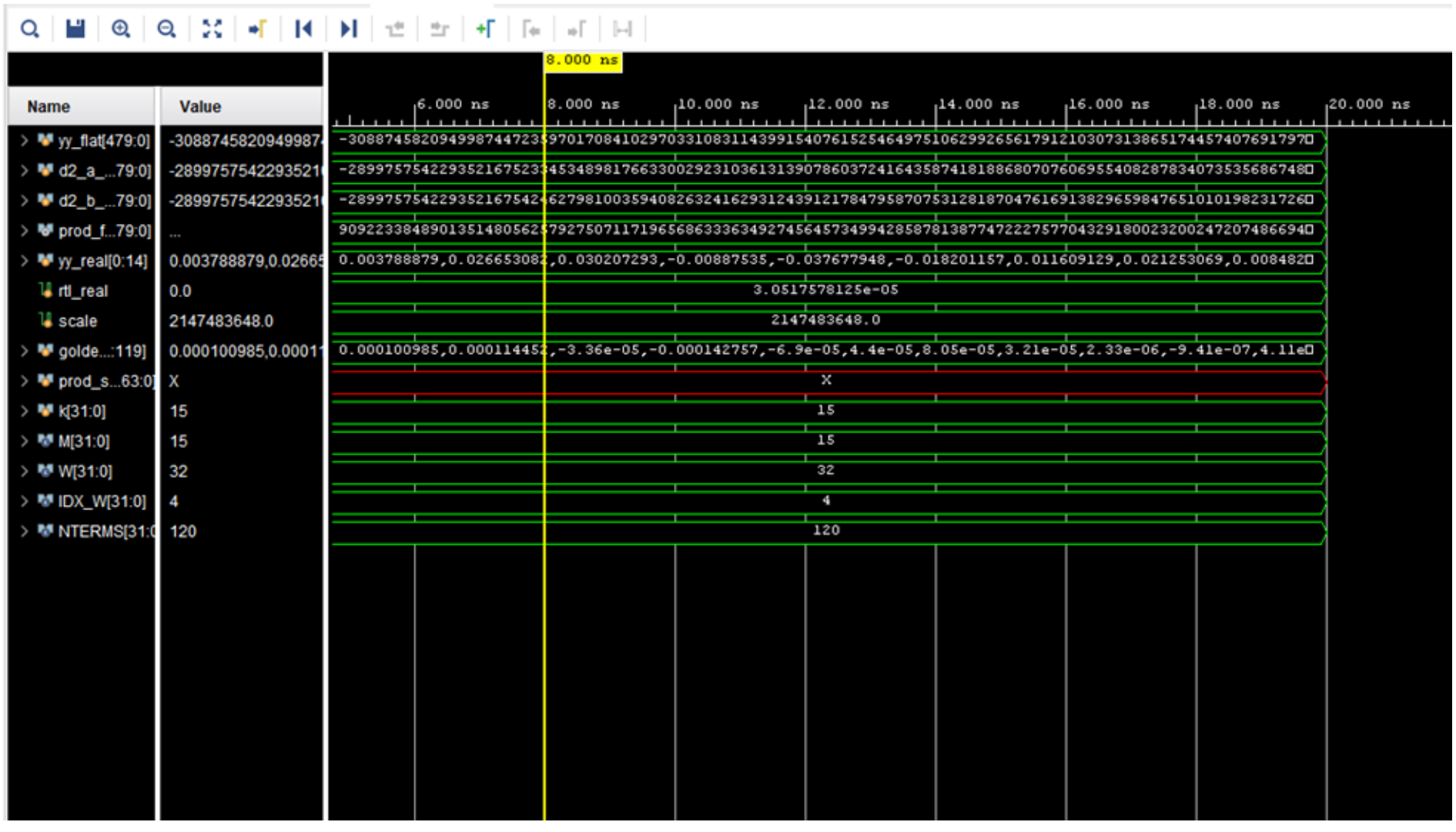
- yy → 15x1 double (column vector)
- Select the elements of yy whose indices are stored in the second cell of d, then multiply all of those selected elements together to produce a single scalar.
- The second parameter ensures that it is done row wise across the columns.
- Finally, produces a column vector as the output.

k	RTL output (float)	Experimental output (float)
0	0.0001009853	0.0001009850
1	0.0001144518	0.0001144520
2	-0.0000336276	-0.0000336000
3	-0.0001427572	-0.0001427570
4	-0.0000689620	-0.0000690000
5	0.0000439856	0.0000440000
6	0.0000805253	0.0000805000
7	0.0000321376	0.0000321000
8	0.0000023323	0.0000023300
9	-0.0000009406	-0.0000009410
10	0.0000410820	0.0000411000
11	0.0000031586	0.0000031600
12	-0.0000342568	-0.0000343000
13	-0.0000801111	-0.0000801000
14	0.0008051174	0.0008051170
15	-0.0002365554	-0.0002365550
16	-0.0010042334	-0.0010042330
17	-0.0004851169	-0.0004851170
18	0.0003094191	0.0003094190
19	0.0005664598	0.0005664600
20	0.0002260738	0.0002260740
21	0.0000164066	0.0000164000
22	-0.0000066164	-0.0000066200
23	0.0002889936	0.0002889940
24	0.0000222194	0.0000222000
25	-0.0002409816	-0.0002409820
26	-0.0005635460	-0.0005635460
27	-0.0002681003	-0.0002681000
28	-0.0011381488	-0.0011381490
29	-0.0005498077	-0.0005498080
30	0.0003506803	0.0003506800
31	0.0006419977	0.0006419980
32	0.0002562209	0.0002562210
33	0.0000185944	0.0000186000
34	-0.0000074987	-0.0000075000
35	0.0003275311	0.0003275310
36	0.0000251823	0.0000252000
37	-0.0002731168	-0.0002731170
38	-0.0006386953	-0.0006386950
39	0.0003344050	0.0003344050
40	0.0001615416	0.0001615420
41	-0.0001030351	-0.0001030350
42	-0.0001886284	-0.0001886280
43	-0.0000752815	-0.0000753000
44	-0.0000054633	-0.0000054600
45	0.0000022032	0.0000022000
46	-0.0000962335	-0.0000962000
47	-0.0000073989	-0.0000074000
48	0.0000802457	0.0000802000
49	0.0001876581	0.0001876580
50	0.0006857822	0.0006857820
51	-0.0004374081	-0.0004374080
52	-0.0008007720	-0.0008007720
53	-0.0003195876	-0.0003195880
54	-0.0000231931	-0.0000232000
55	0.0000093533	0.0000093500
56	-0.0004085338	-0.0004085340
57	-0.0000314102	-0.0000314000
58	0.0003406621	0.0003406620
59	0.0007966530	0.0007966530
60	-0.0002112996	-0.0002113000
61	-0.0003868304	-0.0003868300
62	-0.0001543838	-0.0001543840
63	-0.0000112039	-0.0000112000
64	0.0000045183	0.0000045200
65	-0.0001973512	-0.0001973510
66	-0.0000151734	-0.0000152000
67	0.0001645643	0.0001645640
68	0.0003848407	0.0003848410
69	0.0002467296	0.0002467300

70	0.0000984696	0.0000985000
71	0.0000071461	0.0000071500
72	-0.0000028819	-0.0000028800
73	0.0001258753	0.0001258750
74	0.0000096780	0.0000096800
75	-0.0001049630	-0.0001049630
76	-0.0002454605	-0.0002454600
77	0.0001802704	0.0001802700
78	0.0000130826	0.0000131000
79	-0.0000052759	-0.0000052800
80	0.0002304424	0.0002304420
81	0.0000177176	0.0000177000
82	-0.0001921579	-0.0001921580
83	-0.0004493695	-0.0004493700
84	0.0000052212	0.0000052200
85	-0.0000021056	-0.0000021100
86	0.0000919694	0.0000920000
87	0.0000070711	0.0000070700
88	-0.0000766901	-0.0000767000
89	-0.0001793431	-0.0001793430
90	-0.0000001528	-0.0000001530
91	0.0000066744	0.0000066700
92	0.0000005132	0.0000005130
93	-0.0000055655	-0.0000055700
94	-0.0000130153	-0.0000130000
95	-0.0000026916	-0.0000026900
96	-0.0000002069	-0.0000002070
97	0.0000022445	0.0000022400
98	0.0000052488	0.0000052500
99	0.0000090391	0.0000090400
100	-0.0000980341	-0.0000980000
101	-0.0002292570	-0.0002292570
102	-0.0000075374	-0.0000075400
103	-0.0000176265	-0.0000176000
104	0.0001911694	0.0001911690
105	0.0000143556	0.0000144000
106	0.0007103868	0.0007103870
107	0.0009124805	0.0009124810
108	0.0000787718	0.0000788000
109	0.0014196278	0.0014196280
110	0.0003312821	0.0003312820
111	0.0001347719	0.0001347720
112	0.0004516929	0.0004516930
113	0.0000719458	0.0000719000
114	0.0000003789	0.0000003790
115	0.0000000616	0.0000000616
116	0.0001175659	0.0001175660
117	0.0000006950	0.0000006950
118	0.0000817472	0.0000817000
119	0.0004470580	0.0004470580

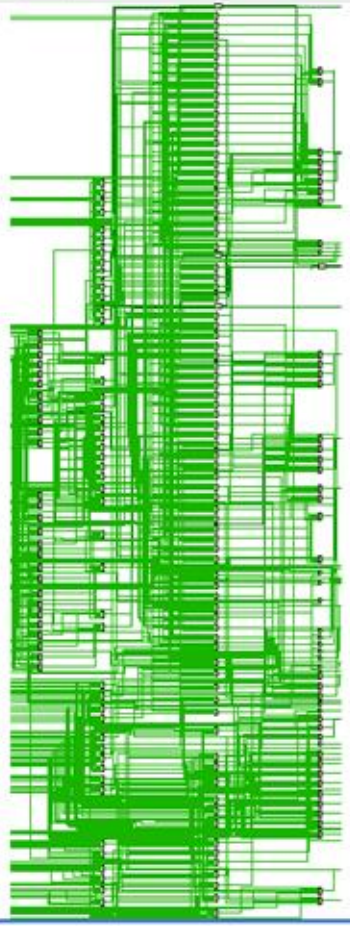
Mean Absolute Error: $6.9275 * 10^{-9}$

Root Mean Squared Error: $1.41216 * 10^{-8}$


8.000 ns
Name
Value
6.000 ns
8.000 ns
10.000 ns
12.000 ns
14.000 ns
16.000 ns
18.000 ns
20.000 ns
yy_flat[479:0]
-3088745820949987
d2_a_...79:0]
-2899757542293521
d2_b_...79:0]
-2899757542293521
prod_f...79:0]
...
yy_real[0:14]
0.003788879,0.02665
rtl_real
0.0
scale
2147483648.0
golde...:119]
0.000100985,0.00011
prod_s...63:0]
X
k[31:0]
15
M[31:0]
15
W[31:0]
32
IDX_W[31:0]
4
NTERMS[31:0]
120
0.003788879,0.026653082,0.030207293,-0.00887535,-0.037677948,-0.018201157,0.011609129,0.021253069,0.008482
3.0517578125e-05
2147483648.0
0.000100985,0.000114452,-3.36e-05,-0.000142757,-6.9e-05,4.4e-05,8.05e-05,3.21e-05,2.33e-06,-9.41e-07,4.11e
X
15
15
32
4
120

Project Summary | Device | volterra_quad.v | volterra_quad_tb.v | **Schematic**

210 Cells 412 Nets 1 of 750



Tcl Console | Messages | Log | Reports | Design Runs | Utilization

- Hierarchy
- Summary
- Slice Logic
  - Slice LUTs (>100%)
    - LUT as Logic (>100%)
  - F7 Muxes (17%)
- Memory
- DSP
  - DSPs (100%)
    - DSP48E1 only
- IO and GT Specific
  - Bonded IOB (>100%)
- Clocking
- Specific Feature
- Primitives
- Black Boxes
- Instantiated Netlists

Hierarchy

Name	Slice LUTs (63400)	F7 Muxes (31700)	DSPs (240)	Bonded IOB (210)
volterra_quad	149406	5428	240	9120

utilization_1

Work Summary - Present

Product(3 column) Function:

- Modified the product(2 column) function (design and testbench) for this function.
- Debugging the code and testbench since the error between the experimental and obtained values is very high.